

Práctica 1.- Programación Orientada a Objetos.



CLASE



OBJETO

Estructura de un programa

- ▣ Un programa corto puede incluirse en una clase y sólo tener el main().

Ejemplo:

```
class Nombre
{
    public static void main(String args[]){
        System.out.println("\n Hola!!! \n Estamos en el
        metodo principal main\n");
    }
}
```

Conceptos de la Orientación a Objetos

⇒ **Clases:** patrones que indican cómo construir los objetos

⇒ **Objetos:** instancias de las clases en tiempo de ejecución

Ejemplo: plano de arquitecto vs edificios

CLASE
Atributos
Métodos

⇒ **Miembros de la clase:**

- × **Atributos:** características o propiedades de los objetos. Pueden ser variables numéricas o referencias a otros objetos

- × **Métodos:** comportamiento de los objetos. Son funciones que operan sobre los atributos de los objetos

CLASES

❖ Atributos:

Una propiedad es una característica que posee un objeto la cual define su apariencia y afecta su comportamiento.

❖ Método:

Un método es un comportamiento que puede tomar un objeto, el cual es provocado por el mundo que rodea al objeto.



Atributos

Métodos

¿Cómo definir una clase?

Definición de Clase

⇒ Sintaxis:

```
class <NombreClase>
{
    // Declaración de atributos
    <tipo> <variable> ;

    // Declaración de métodos
    <tipo> <nombreMétodo> ( <argumentos> )
    { ... }
}
```

El nombre de la clase nos permite:

1. Declarar variables de ese tipo.
2. Como para crear objetos del mismo.

El nombre del archivo Java debe coincidir con el nombre de la clase definida en él.

<NombreClase>.java

Reglas para elegir un nombre de una Clase

- El programador elige los nombres de las clases, objetos y métodos. Debe hacerlo lo más significativo posible.
- Los nombres formados por letras (mayúsculas y minúsculas) y dígitos (0-9).
- Siempre deben empezar con una letra.
- En JAVA por convención los nombres de las clases empiezan con una letra mayúscula.
- Los nombres de los objetos y métodos empiezan con letras minúsculas.

Estructura de una Clase

acceso class <nombre de la clase>

{

// atributos

acceso tipo variable-1 ;

acceso tipo variable-2;

....

acceso tipo variable-n;

//métodos

acceso tipo <nombre_metodo1>(lista de parámetros){

cuerpo del metodo

}

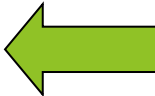
....

acceso tipo <nombre_metodo2>(lista de parámetros){

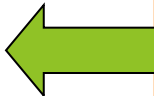
cuerpo del metodo

}

} //fin de la clase



Atributos: Campos (datos)



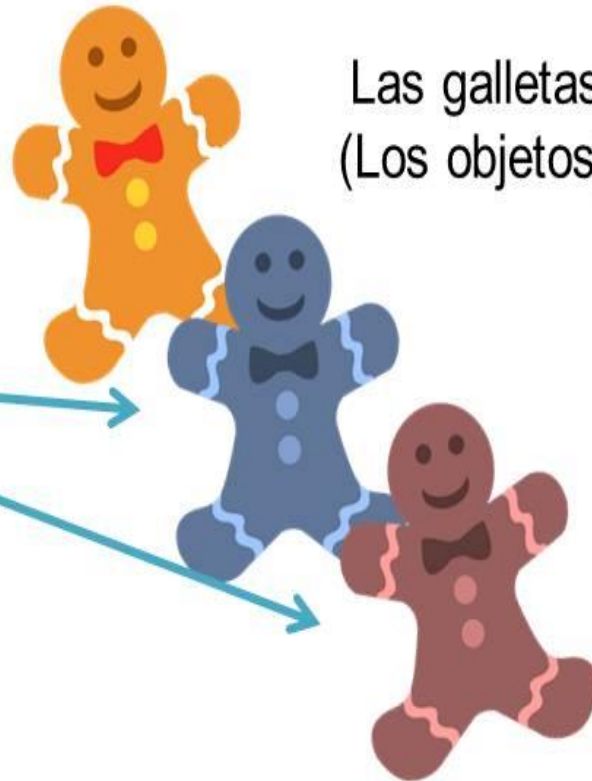
**Métodos: Comportamiento
(funciones y
procedimientos)**

OBJETOS

Cortador de galletas.
(La Clase)



Las galletas
(Los objetos)



¿Cómo creamos un objeto?

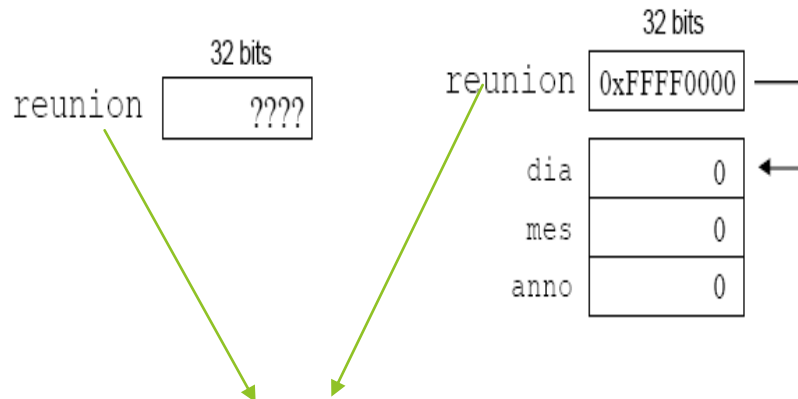
Creación de un Objeto

⇒ Se utiliza la palabra reservada `new`

```
<refObjeto> = new <NombreClase>() ;
```

⇒ Ejemplo: `Fecha reunion ;`
`reunion = new Fecha ( ) ;`

La declaración no crea el objeto. Para crear el objeto de la clase se necesita utilizar el operador **NEW**, con propósito de indicarle a la computadora que cree un objeto y asigne espacio de memoria para ella.



Una variable de **tipo clase**: es una variable **referencia**, que puede contener la dirección en memoria(o referencia) de un objeto de la clase o null para una referencia no válida.

Pasos para crear Objetos

1. Creación de la clase

```
class Fecha  
{  
    // declaración de  
    //variables  
    //declaración de los  
    //métodos  
}
```

2. Declarar los Objetos

```
Fecha reunion;
```

3. Crear los Objetos

```
reunion = new Fecha ( );
```

¿Cómo acceder a los atributos y métodos del objeto?

Una vez creado un objeto, se puede acceder a sus datos y métodos utilizando el operador **punto** (.):

Ejemplo:

nombreobjeto.datos Referencia a un dato de un objeto.

nombreobjeto.método() Referencia a un método de un objeto.

Ejemplo:

```
Fecha reunion = new Fecha ( ) ;  
reunion.dia = 15 ;  
reunion.mes = 12 ;  
reunion.anno = 2000 ;  
reunion.darFormato() ;
```

Acceso a los Miembros de un Objeto

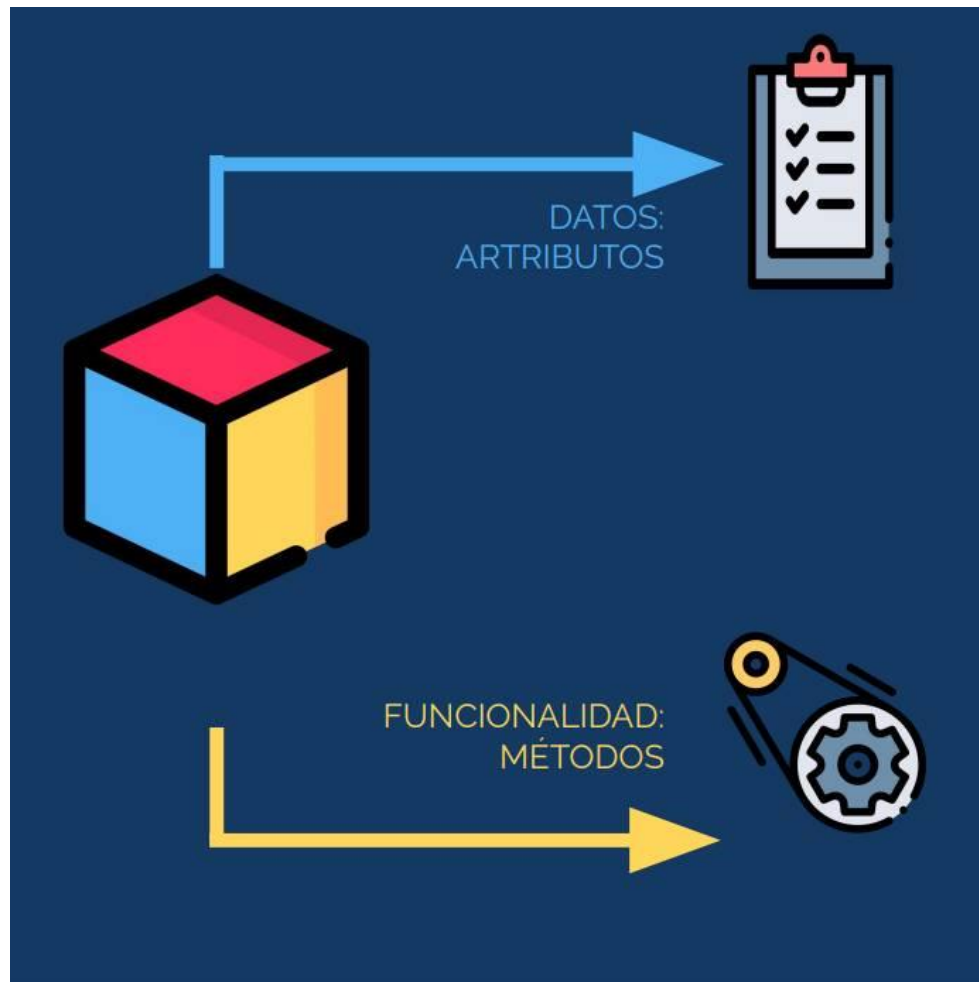
⇒ A través del operador **punto** (.) se puede acceder tanto a los atributos como a los métodos

`<refObjeto>.<atributo>   ó   <refObjeto>.<método>()`

Ejemplo:

```
Fecha reunion = new Fecha ( ) ;  
reunion.dia = 15 ;  
reunion.mes = 12 ;  
reunion.anno = 2000 ;  
reunion.darFormato() ;
```

MÉTODOS



MÉTODOS

- ❖ Los métodos son las acciones que realizan un objeto de una clase.
- ❖ Los métodos son bloques de código (subprogramas), definidos dentro de una clase.
- ❖ Una invocación a un método es una **petición al método** para que ejecute su acción y lo haga con el objeto mencionado.

MÉTODOS

- ❖ La invocación de un método se denominaría también **llamar a un método** y pasar un **mensaje** a un objeto.
- ❖ Existen dos tipos de métodos, aquellos que devuelven un **valor único (Función)** y aquellos que ejecutan alguna acción distinta de devolver de forma explícita un único valor.
- ❖ Los métodos que realizan alguna acción distinta de devolver un valor explícito se denominan **métodos void (Procedimientos)**

DEFINICIÓN DE MÉTODOS

```
<tipoRetorno> <nombreMétodo> ( <listaArgumentos> )  
{  
    <bloqueCódigo>  
}
```

⇒ **<tipoRetorno>**: tipo de dato que retorna el método
(primitivo o referencia)

Si no devuelve ningún valor, debe ser `void`

⇒ **<nombreMétodo>**: identificador del método

⇒ **<listaArgumentos>**: el método admite que le pasen
argumentos separados por comas con el formato:

```
[<tipo> <argumento> [, <tipo> <argumento>...]]
```


Ejemplo de Métodos

```
⇒ double tangente ( double x )  
{  
    return Math.sin(x) / Math.cos(x) ;  
}
```

```
⇒ void imprimirHola ( )  
{  
    System.out.println ( "Hola" ) ;  
}
```

```
⇒ String darFormato ( int dia , int mes , int anno )  
{  
    String s ;  
  
    s = dia + "/" + mes + "/" + anno ;  
    return s ;  
}
```